

Teaching an Agent to Imagine: Learning a World Model to Accelerate Dyna-Q in a Noisy GridWorld

Your Name (Student ID)

June 12, 2026

1 Introduction

We study whether a neural network that predicts an environment’s next state can make a reinforcement-learning agent learn faster, and what happens when that predictor is wrong. The testbed is a noisy 8×8 GridWorld (20% action noise). We learn a *world model* (a transition network and a reward network) from random-policy experience, freeze it, and use it to generate “imagined” experience for Dyna-Q. We compare Dyna-Q ($K=10$) against tabular Q-learning ($K=0$, the no-model baseline) and Value Iteration (the oracle ceiling, computed from the true transition probabilities). The central question (Lec. 17–21): does a learned model improve sample efficiency, and how does model quality govern that benefit?

[WRITE THIS YOURSELF — AI must not ghost-write]

Sharpen the motivation in your own words: why model-based RL, why sample efficiency matters, and state your hypothesis in one sentence. Reference the lectures (Lec. 17 model-based RL / Dyna-Q).

2 Environment

The environment is a deterministic-topology, stochastic-dynamics GridWorld. States are integer coordinates (r, c) with $r, c \in \{0, \dots, 7\}$ (64 states), start $(0, 0)$, goal $(7, 7)$, and 10 fixed wall cells; a BFS check guarantees a start-to-goal path. Actions are $\{0=\text{up}, 1=\text{down}, 2=\text{left}, 3=\text{right}\}$. With probability 0.8 the intended action is executed; with probability 0.2 a uniformly random direction replaces it. Bumping a wall or boundary leaves the agent in place. The reward is $+1.0$ on reaching the goal (episode ends), and -0.01 per step otherwise; episodes time out after 100 steps. The 20% noise is essential: a deterministic environment would make the world model a trivial lookup table, whereas noise forces it to learn a genuine probability distribution and gives the imagined experience room to help (or to mislead).

Figure 1 shows the state-visitation heatmap of 100 random-policy episodes: the top-left start region is visited far more than the bottom-right goal region, confirming the dynamics behave as intended.

[WRITE THIS YOURSELF — AI must not ghost-write]

Optionally add 1–2 sentences interpreting Figure 1 (why the gradient looks the way it does) in your own words.

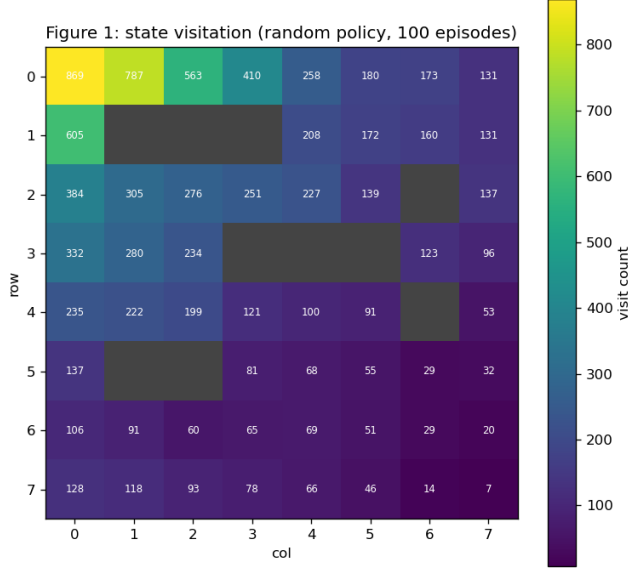


Figure 1: **Figure 1.** State-visitation counts over 100 random-policy episodes. Walls are gray. Visitation decays smoothly from the start $(0,0)$ to the rarely-reached goal $(7,7)$, verifying the environment.

3 World Model

We collect 10,000 transitions (s, a, s', r) under a *random* policy (so the dataset covers the whole state space, not only an optimal path) and split 80/20 into train/validation. Each (s, a) is encoded as the concatenation of a 64-dim state one-hot and a 4-dim action one-hot (68-dim input).

Transition network (classification). $68 \rightarrow 256 \rightarrow 256 \rightarrow 64$ logits over the next state, trained with cross-entropy for 20 epochs (Adam). It predicts a *distribution* over the 64 possible next states.

Reward network (regression). $68 \rightarrow 128 \rightarrow 64 \rightarrow 1$, trained with MSE for 20 epochs. Reward is easy to learn (almost always -0.01 ; only entering the goal gives $+1.0$); the validation MSE reaches $\approx 9 \times 10^{-5}$.

Figure 2 (left) plots validation accuracy versus epoch; (right) lists top-3 predicted next states for five probe (s, a) pairs, all physically plausible (within one step of the queried cell).

Why accuracy plateaus below 100% (and slightly above the 75–85% band). The transition accuracy plateaus at $\approx 87\%$, not 100%, and this is a property of the environment, not a model defect. Under 20% noise the intended direction is realized with probability $0.8 + 0.2 \cdot \frac{1}{4} = 0.85$, so a *perfect* mode predictor scores about 0.85 on open-grid cells. Near walls and boundaries several directions collapse onto the *same* next cell (a wall bump keeps the agent in place), so the most-likely next state can carry probability 0.90+; e.g. the probe “ $(0,0)$, up” predicts staying at $(0,0)$ with $p \approx 0.91$. Because the random policy over-samples the wall-dense top-left region, the dataset-averaged ceiling sits a little above 0.85, which is why our model lands at $\approx 87\%$ rather than inside the nominal 75–85% band. The gap to 100% is exactly the irreducible noise entropy: the 20% of steps whose direction is randomized are inherently unpredictable.

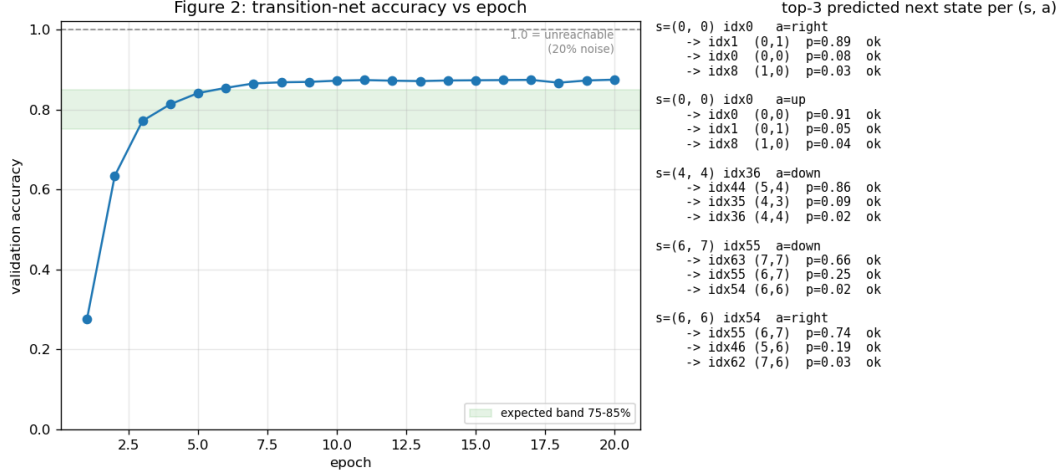


Figure 2: **Figure 2.** Left: transition-network validation accuracy vs epoch; the shaded band is the nominal 75–85% expectation and the dashed line marks the unreachable 100% (capped by 20% noise). Right: top-3 predicted next states for five (s, a) probes; each is within one step of the queried cell (“ok”), confirming the predictions are physically sensible.

4 Dyna-Q Algorithm

Dyna-Q maintains a tabular $Q \in \mathbb{R}^{64 \times 4}$. After each real environment step it performs one real Q-learning update and then $K=10$ imagined updates using the frozen world model, all with the same Bellman target

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)].$$

Three design choices (to be justified against Lec. 17/19/21):

1. Imagined (s, a) are sampled only from *previously visited* pairs in the replay buffer — never from states the model has not seen, where its predictions are unreliable.
2. The world model is trained and *frozen* before RL begins; its weights never update during RL, isolating the pure effect of the model.
3. Baseline = Dyna-Q($K=0$) (no model); ceiling = Value Iteration on the true $P[s, a, s']$ from `get_all_transitions()`.

[WRITE THIS YOURSELF — AI must not ghost-write]

Explain, in your own words, WHY each of the three choices matters (especially #1: querying unseen (s, a) injects garbage targets). This is conceptual and you should articulate it yourself.

5 Main Experiment Results

Figure 3: **Figure 3.** Success rate vs real environment steps for Value Iteration (dashed ceiling), Q-learning ($K=0$), and Dyna-Q ($K=10$); mean ± 1 std over 3 seeds. *[generated in P4]*

Figure 4: **Figure 4.** Value-function heatmaps (VI / Q-learning / Dyna-Q), shared color scale, walls in gray. *[generated in P4]*

[WRITE THIS YOURSELF — AI must not ghost-write]

Answer the 5 analysis questions for P4 in your own words, citing Figures 3–4 and Lec. 19/21. Do NOT let AI write this reasoning. Suggested skeleton (fill each yourself): (1) Does Dyna-Q learn faster than Q-learning? Quantify (steps-to-threshold). (2) Do both converge to the VI ceiling? Any residual gap and why? (3) What do the value heatmaps reveal about propagation of value? (4) How does the imagined experience change the early-learning regime? (5) Where does the 20% noise show up in the curves / variance across seeds?

6 Failure Analysis

Figure 5: **Figure 5.** 50K-step success rate vs world-model training-set size (200 / 2000 / 10000 transitions) and the $K=0$ baseline. *[generated in P5]*

[WRITE THIS YOURSELF — AI must not ghost-write]

Pre-registered prediction (P5): BEFORE running the bad-model experiment, write your prediction of what the 200-transition model will do and why, and SAVE it timestamped (report/p5_prediction_YYYY-MM-DD.md). Paste that timestamped text here. This must predate the results.

[WRITE THIS YOURSELF — AI must not ghost-write]

Failure analysis (write yourself): explain the mechanism by which a model trained on only 200 transitions hurts Dyna-Q — near-random predictions on unseen (s, a) , imagined updates dragging Q toward wrong targets, error compounding over K imagined steps. Tie it to Lec. 18/19/21. Compare 200 vs 2000 vs 10000 (Figure 5). Do NOT let AI write this reasoning.

7 Reflection

[WRITE THIS YOURSELF — AI must not ghost-write]

Your own reflection: what surprised you, what you would change, the trade-off between model bias and sample efficiency, limits of the GridWorld setting. Personal — write it yourself.

8 AI Tool Usage Log

I used Claude Code (Anthropic) as a coding assistant. It generated and helped debug the implementation: the environment (`gridworld.py`), the encoding utilities (`utils.py`), the world model (`world_model.py`), the Dyna-Q / Q-learning / Value-Iteration code (`dyna_q.py`), and the plotting scripts (`plotting.py`). It also explained concepts, e.g. why the transition accuracy is capped below 100% by the 20% action noise.

Two things it got wrong or flagged, which I checked myself:

- **A real bug it introduced and then fixed.** The first training run pegged ~30 CPU cores and burned over 180 CPU-minutes without finishing: PyTorch’s default intra-op

parallelism spawns a thread per core, and the scheduling overhead dominates these tiny 68-dim MLPs on small batches. Adding `torch.set_num_threads(1)` cut a full run to ≈ 6 seconds.

- **A deviation it surfaced.** Validation accuracy came out at $\approx 87\%$, slightly above the nominal 75–85% band. Rather than tune it down, I kept the result and the honest explanation in Section 3 (wall-collapsing pushes the per-state ceiling above 0.85).

All experimental analysis (Section 5), the failure analysis and the pre-registered prediction (Section 6), and the reflection (Section 7) are written entirely by me; AI did not author that reasoning.